

std::function_wrapper

Document #: P3843R2
Date: 2025-11-17
Project: Programming Language C++
Audience: LEWG
Reply-to: Jonathan Müller
<foonathan@jonathanmuller.dev>

Contents

- [1 Abstract](#)
- [2 Revision history](#)
 - [2.1 R2](#)
 - [2.2 R1](#)
- [3 Background](#)
- [4 Proposal](#)
- [5 Wording](#)
 - [5.1 Option 1: C++26 target](#)
 - [5.2 Option 2: C++29 target](#)
- [6 Acknowledgments](#)
- [7 References](#)

§ 1 Abstract

Introduce `std::function_wrapper`, a callable type that encodes a compile-time known function into the type system, as originally proposed by [\[P3774R0\]](#).

§ 2 Revision history

§ 2.1 R2

- Change title to reflect current direction.
- No longer proposed as a full replacement of `std::nontype_t` (now `std::constant_arg`).

§ 2.2 R1

- Update wording to allow copy elision and make it freestanding.
- Reference prior implementation in libstdc++.

§ 3 Background

[P0792R14], adopted for C++26, adds `std::function_ref`, a non-owning reference wrapper for callable objects. To make it usable with member function pointers, it has an overload that takes a member function pointer using a `std::nontype_t` parameter. That way, we can encode the member function into the type system, making it possible to reference it without needing to keep the member function pointer alive.

Meanwhile, [P2841R1] renamed the core language term “non-type template parameter” to “constant template parameter”, so the naming of `std::nontype_t` should be reconsidered.

It also seemed that the `std::nontype_t` utility has become redundant with the adoption of [P2781R9]’s `std::constant_wrapper`. Like `std::nontype_t`, it allows encoding of a value into the type system, but it has features that make it more ergonomic to use. For example, it overloads all operators of the underlying type, automatically re-wrapping the result in a `std::constant_wrapper` again.

However, as [P3792R0] points out, `std::constant_wrapper` is not a good replacement for `std::nontype_t` due to the fact that `std::constant_wrapper` is a callable itself. It overloads `operator()` with the same semantics as other operators: require that the operands are `std::constant_wrapper`’s themselves, unwrap them, forward to the underlying values, and re-wrap the result. This means that, as of the C++26 working draft, given `int fn(int)`, we have the following options to construct a `std::function_ref`:

- `function_ref(fn)` creates a function ref to `fn` with signature `int(int)`
- `function_ref(std::nontype<fn>)` creates a function ref to `fn` with signature `int(int)`
- `function_ref(std::cw<fn>)` creates a function ref to `std::cw<fn>` with signature `std::cw<int>(std::cw<int>)` (and also stores a dangling pointer to the `std::cw` object, but that’s not the point here)

Therefore, using `std::constant_wrapper` instead of `std::nontype_t` would change the semantics of the function `std::constant_wrapper` naturally represents: Instead of a function that takes and returns `std::constant_wrapper`’s, it’s a function that takes and returns the underlying type. In particular, it would be inconsistent with the behavior in e.g. `std::function`.

As such, [P3774R0] proposed to rename `std::nontype_t` to `std::fn_t` in C++26 and give it a call operator in C++29. This approach did not have consensus, so following

LEWG guidance [P3774R1] pivoted to rename it to `std::constant_arg_t` instead without a call operator. This was then subsequently adopted.

§ 4 Proposal

The idea behind [P3774R0] is nice: Renaming `std::nontype_t` to something like `std::fn_t` and making it callable gives it an identity distinct from `std::constant_wrapper`: `std::fn_t` is a compile-time known function, while `std::constant_wrapper` is a generic compile-time value. Furthermore, `std::fn_t` with a call operator is broadly useful beyond `std::function_ref`:

- It can be used to specify a function as the comparison or hasher of a `std::[unordered_]map`. Currently, you have to use a lambda to avoid the overhead of storing the pointer. With `std::fn_t` you just specify the function as template parameter.
- When passing a function to generic algorithms, unless the entire algorithm is inlined, we suffer an indirect call. If we pass a `std::fn_t` instead, the call target is known at compile time, enabling further information.
- `libstdc++` already uses a type like `std::fn_t` as a QoI optimization for the return type of `std::bind_front/back<f>()` (i.e. when binding zero arguments to a compile-time known function) [libstdc++].

Even though it failed to gain consensus when originally proposed and when discussing R1 of this paper, `std::fn_t` on its own is still useful. I therefore propose it for inclusion in C++29 under the name `std::function_wrapper`, as that had the most consensus during the LEWG telecon. Note that even though we do not propose removal of the `std::constant_arg_t` constructor, `std::function_ref` should still special case `std::function_wrapper`; it would be silly to store a reference to a `std::function_wrapper` object as opposed to the underlying function!

However, this special cased constructor template `<auto fn> function_ref(function_wrapper<fn>)` would do exactly the same as template `<auto fn> function_ref(constant_arg_t<fn>)`; the only difference is the spelling of the way the argument is being passed. The objections around replacing `std::nontype_t/std::constant_arg_t` with `std::function_wrapper` focused on the other constructor taking a `std::nontype_t/std::constant_arg_t` and a bound entity, not the unary constructor. I do not think it makes sense to have both a `std::constant_arg_t` and a `std::function_wrapper` constructor that do the same thing, so if we adopt this in C++26, we should replace it.

§ 5 Wording

Adapted from [P3774R0] with feedback from the discussion on [LWG4319], relative to [N5008].

In [version.syn], update the feature-test macros:

```
#define __cpp_lib_function_wrapper 20XXXXL // also in <functional>
```

In [functional.syn], change the synopsis as follows:

```
namespace std {
    [...]

    // [func.identity], identity
    struct identity; // freestanding

+ // [func.wrapper], constant function wrapper
+ template<auto f> struct function_wrapper; // freestanding
+ template<auto f> constexpr function_wrapper<f> fw; // freestanding

    // [func.not.fn], function template not_fn
    template<class F> constexpr unspecified not_fn(F&& f); // freestanding
    template<auto f> constexpr unspecified not_fn() noexcept; // freestanding

    [...]
}
```

Between [func.identity] and [func.not.fn], insert a new subclause:

§ Constant function wrapper [func.wrapper]

```
template<auto f>
struct function_wrapper {
    explicit function_wrapper() = default;

    see below
};
```

- 1 Let fw be an object of type FW that is a (possibly const) specialization of function_wrapper, and let cf be a template parameter object ([temp.param]) corresponding to the constant template argument of FW. Then:
 - FW is a trivially copyable type, such that FW models semiregular and is_empty_v<FW> is true;
 - fw is a simple call wrapper ([func.require]) with no state entities and with the call pattern invoke(cf, call_args...), where call_args is an argument pack used in a function call expression ([expr.call]),
 - for any type R and pack of types Args, both fw and std::move(fw) are convertible to:
 - R(*) (Args...) if is_invocable_r_v<R, decltype(cf), Args...> is true;

- `R(*) (Args...) noexcept` if `is_nothrow_invocable_r_v<R, decltype(cf), Args...>` is true.

[*Example:*

```
bool is_even(long);
bool(*ptr)(int) = std::fw<is_even>;
```

— *end example*]

Note that the above wording for simple call wrappers does not include the line “except that any parameter of the function selected by overload resolution may be initialized from the corresponding element of call_args if that element is a prvalue” present in [P3774R0]. This enables copy elision in the calls to fw (by implementing it as a surrogate function call).

In [func.wrap.ref.ctor], update the constructor in paragraph 5-7 to not be considered for function_wrapper:

```
template<class F> constexpr function_ref(F&& f) noexcept;
```

5 Let T be `remove_reference_t<F>`.

6 *Constraints:*

(6.1) — `remove_cvref_t<F>` is not the same type as `function_ref`,

? — `remove_cvref_t<F>` is not a specialization of `function_wrapper`,

(6.2) — `is_member_pointer_v<T>` is false, and

(6.3) — `is_invocable_using<cv T&>` is true.

7 *Effects:* Initializes `bound-entity` with `addressof(f)`, and `thunk_ptr` with the address of a function `thunk` such that `thunk(bound-entity, call_args...)` is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(static_cast<cv T&>(f), call_args...)`.

§ 5.1 Option 1: C++26 target

If adoption for C++26, replace the `nontype_t` constructor.

In [func.wrap.ref.class], replace the constructor in the synopsis:

```
// [func.wrap.ref.ctor], constructors and assignment operators
template<class F> function_ref(F*) noexcept;
template<class F> constexpr function_ref(F&&) noexcept;
- template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
+ template<auto f> constexpr function_ref(function_wrapper<f>) noexcept;
template<auto f, class U> constexpr function_ref(nontype_t<f>, U&&)
    noexcept;
template<auto f, class T> constexpr function_ref(nontype_t<f>, cv T*)
    noexcept;
```

In [func.wrap.ref.ctor], update the constructor in paragraph 8-11 to take `function_wrapper`:

```
-template<auto f> constexpr function_ref(nontype_t<f>) noexcept;  
+template<auto f> constexpr function_ref(function_wrapper<f>) noexcept;
```

- 8 Let F be `decltype(f)`.
- 9 *Constraints:* `is-invocable-using` is true.
- 10 *Mandates:* If `is_pointer_v<F> || is_member_pointer_v<F>` is true, then `f != nullptr` is true.
- 11 *Effects:* Initializes *bound-entity* with a pointer to an unspecified object or null pointer value, and *thunk_ptr* with the address of a function *thunk* such that *thunk(bound-entity, call-args...)* is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(f, call-args...)`.

§ 5.2 Option 2: C++29 target

If adoption for C++29, add an additional `function_wrapper` constructor.

In [func.wrap.ref.class], add the constructor in the synopsis:

```
// [func.wrap.ref.ctor], constructors and assignment operators  
template<class F> function_ref(F*) noexcept;  
template<class F> constexpr function_ref(F&&) noexcept;  
template<auto f> constexpr function_ref(nontype_t<f>) noexcept;  
+template<auto f> constexpr function_ref(function_wrapper<f>) noexcept;  
template<auto f, class U> constexpr function_ref(nontype_t<f>, U&&) noexcept;  
template<auto f, class T> constexpr function_ref(nontype_t<f>, cv T*) noexcept;
```

In [func.wrap.ref.ctor], add the new constructor after paragraph 8-11:

```
template<auto f> constexpr function_ref(function_wrapper<f>) noexcept;
```

- ? Let F be `decltype(f)`.
- ? *Constraints:* `is-invocable-using` is true.
- ? *Mandates:* If `is_pointer_v<F> || is_member_pointer_v<F>` is true, then `f != nullptr` is true.
- ? *Effects:* Initializes *bound-entity* with a pointer to an unspecified object or null pointer value, and *thunk_ptr* with the address of a function *thunk* such that *thunk(bound-entity, call-args...)* is expression-equivalent ([defns.expression.equivalent]) to `invoke_r<R>(f, call-args...)`.

§ 6 Acknowledgments

Thanks to Tomasz Kamiński for providing feedback and suggesting the wording change to allow copy elision.

§ 7 References

[libstdc++] `_Bind_fn_t` in libstdc++.

<https://github.com/gcc-mirror/gcc/blob/27861393a92d00f8ab7b0f139075ad43dd418282/libstdc%2B%2B-v3/include/std/functional#L926-L936>

[LWG4319] Tomasz Kamiński. Supporting copy-elision in function wrappers.

<https://wg21.link/lwg4319>

[N5008] Thomas Köppe. 2025-03-15. Working Draft, Programming Languages — C++.

<https://wg21.link/n5008>

[P0792R14] Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo. 2023-02-09. `function_ref`: a non-owning reference to a Callable.

<https://wg21.link/p0792r14>

[P2781R9] Zach Laine, Matthias Kretz, Hana Dusíková. 2025-06-17. `std::constexpr_wrapper`.

<https://wg21.link/p2781r9>

[P2841R1] Corentin Jabot, Gašper Ažman. 2023-10-14. Concept Template Parameters.

<https://wg21.link/p2841r1>

[P3774R0] Jan Schultke, Bronek Kozicki, Tomasz Kamiński. 2025-07-15. Rename `std::nontype`, and make it broadly useful.

<https://wg21.link/p3774r0>

[P3774R1] Jan Schultke, Bronek Kozicki, Tomasz Kamiński. 2025-08-14. Rename `std::nontype`, and make it broadly useful.

<https://wg21.link/p3774r1>

[P3792R0] Bronek Kozicki. 2025-07-13. Why ``constant_wrapper`` is not a usable replacement for ``nontype``.

<https://wg21.link/p3792r0>